

Lesson 4–6 Consolidation Practice: Mini Task Manager

Strongly Recommended Practice Before Assignment 1

Purpose

This practice activity helps you review and combine the main skills from Lessons 4–6:

- Jakarta Faces pages
- Managed beans
- PrimeFaces components
- form submission
- validation messages
- `p:dataTable`
- navigation
- JSF lifecycle behavior

This is **not a graded assignment**, but it is strongly recommended before starting Assignment 1. The same patterns appear in upcoming graded work.

What You Will Build

You will build a small **Mini Task Manager** web application.

Your application will allow a user to:

1. Add a task
2. Display tasks in a table
3. Mark tasks as complete
4. Remove tasks
5. Navigate to a summary page

This updated practice keeps the useful task-manager structure from the previous checklist, including command buttons, input text, data table, selection components, feedback messages, and dynamic updates.

Estimated Time

60–90 minutes outside of class.

Files to Create

Create the following files:

```
src/main/java/dmit2015/model/Task.java
src/main/java/dmit2015/view/TaskManagerBean.java
src/main/webapp/task-manager.xhtml
src/main/webapp/task-summary.xhtml
```

Part 1 — Create the Task Model Class

Create:

```
package dmit2015.model;

public class Task {

    private String description;
    private String priority;
    private boolean done;

    public Task(String description, String priority) {
        this.description = description;
        this.priority = priority;
        this.done = false;
    }

    public String getDescription() {
        return description;
    }
}
```

```

    public String getPriority() {
        return priority;
    }

    public boolean isDone() {
        return done;
    }

    public void setDone(boolean done) {
        this.done = done;
    }
}

```

Part 2 – Create the Managed Bean

Create:

```

package dmit2015.view;

import dmit2015.model.Task;
import jakarta.enterprise.context.SessionScoped;
import jakarta.faces.application.FacesMessage;
import jakarta.faces.context.FacesContext;
import jakarta.inject.Named;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

@Named
@SessionScoped
public class TaskManagerBean implements Serializable {

    private String newTaskDescription;
    private String selectedPriority;
    private List<Task> tasks = new ArrayList<>();

    public void addTask() {

        Task newTask = new Task(newTaskDescription, selectedPriority);
        tasks.add(newTask);

        FacesContext.getCurrentInstance().addMessage(null,
            new FacesMessage("Task added: " + newTaskDescription));
    }
}

```

```
        newTaskDescription = "";
        selectedPriority = "";
    }

    public void removeTask(Task task) {
        tasks.remove(task);

        FacesContext.getCurrentInstance().addMessage(null,
            new FacesMessage("Task removed"));
    }

    public String viewSummary() {
        return "task-summary?faces-redirect=true";
    }

    public long getCompletedTaskCount() {
        return tasks.stream()
            .filter(Task::isDone)
            .count();
    }

    public int getTotalTaskCount() {
        return tasks.size();
    }

    public String getNewTaskDescription() {
        return newTaskDescription;
    }

    public void setNewTaskDescription(String newTaskDescription) {
        this.newTaskDescription = newTaskDescription;
    }

    public String getSelectedPriority() {
        return selectedPriority;
    }

    public void setSelectedPriority(String selectedPriority) {
        this.selectedPriority = selectedPriority;
    }

    public List<Task> getTasks() {
        return tasks;
    }
}
```

Part 3 — Create the Task Manager Page

Create:

src/main/webapp/task-manager.xhtml

Use the following starter code:

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core"
      xmlns:p="primefaces">

  <h:head>
    <title>Mini Task Manager</title>
  </h:head>

  <h:body>

    <h2>Mini Task Manager</h2>

    <h:form>

      <p:messages id="messages" />

      <p:panel header="Add New Task">

        <p:outputLabel for="description" value="Task Description:" />
        <br/>

        <p:inputText id="description"
                    value="#{taskManagerBean.newTaskDescription}"
                    required="true"
                    requiredMessage="Task description is required" />

        <br/><br/>

        <p:outputLabel for="priority" value="Priority:" />
        <br/>

        <p:selectOneMenu id="priority"
                      value="#{taskManagerBean.selectedPriority}"
                      required="true"
                      requiredMessage="Priority is required">
```

```

        <f:selectItem itemLabel="Select Priority" itemValue="" />
        <f:selectItem itemLabel="Low" itemValue="Low" />
        <f:selectItem itemLabel="Medium" itemValue="Medium" />
        <f:selectItem itemLabel="High" itemValue="High" />

    </p:selectOneMenu>

    <br/><br/>

    <p:commandButton value="Add Task"
        action="#{taskManagerBean.addTask}"
        update="@form" />

</p:panel>

<br/>

<p:panel header="Task List">

    <p:dataTable value="#{taskManagerBean.tasks}"
        var="currentTask"
        emptyMessage="No tasks have been added yet.">

        <p:column headerText="Description">
            <h:outputText value="#{currentTask.description}" />
        </p:column>

        <p:column headerText="Priority">
            <h:outputText value="#{currentTask.priority}" />
        </p:column>

        <p:column headerText="Done">
            <p:selectBooleanCheckbox value="#{currentTask.done}">
                <p:ajax update="@form" />
            </p:selectBooleanCheckbox>
        </p:column>

        <p:column headerText="Action">
            <p:commandButton value="Remove"
                action="#{taskManagerBean.removeTask(currentTask)}"
                update="@form" />
        </p:column>

    </p:dataTable>

</p:panel>

<br/>

```

```
<p:commandButton value="View Summary"
                 action="#{taskManagerBean.viewSummary}" />

</h:form>

</h:body>
</html>
```

Part 4 — Create the Summary Page

Create:

src/main/webapp/task-summary.xhtml

Your page must display:

- total number of tasks
- number of completed tasks
- number of incomplete tasks
- link or button back to the task manager page

Use this starter code:

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:p="primefaces">

  <h:head>
    <title>Task Summary</title>
  </h:head>

  <h:body>

    <h2>Task Summary</h2>

    <p>Total Tasks:
      <h:outputText value="#{taskManagerBean.totalTaskCount}" />
    </p>

    <p>Completed Tasks:
      <h:outputText value="#{taskManagerBean.completedTaskCount}" />
```

</p>

<p>Incomplete Tasks:

```
<h:outputText value="#{taskManagerBean.totalTaskCount - taskManagerBean.completedTaskCount}"
/>
```

</p>

```
<h:link value="Back to Task Manager"
outcome="task-manager" />
```

</h:body>

</html>

Part 5 – Run and Test

Open the page directly in the browser:

<http://localhost:8080/your-app-name/task-manager.xhtml>

Test the following:

Test	Expected Result
Submit with blank description	Validation message appears
Submit without priority	Validation message appears
Add a valid task	Task appears in table
Add multiple tasks	All tasks appear in table
Mark task as done	Checkbox remains selected
Remove task	Task disappears from table
Click View Summary	Browser navigates to summary page

Test	Expected Result
Click Back to Task Manager	Returns to task manager page

Checkpoint Questions

Be prepared to answer:

1. Why does validation stop the task from being added?
2. Why is `@SessionScoped` used in this version?
3. What does `update="@form"` do?
4. What does `faces-redirect=true` do?
5. Why do we need getters and setters for JSF binding?

Bonus Challenge for Early Finishers

Bonus 1 — Add a Due Date Field

Add a `dueDate` property to `Task`.

Then update:

- constructor
- getter
- form input
- data table column

Suggested component:

```
<p:datePicker value="#{taskManagerBean.selectedDueDate}" />
```

Bonus 2 — Add a Clear Completed Button

Add a method to remove all completed tasks:

```
public void clearCompletedTasks() {  
    tasks.removeIf(Task::isDone);  
  
    FacesContext.getCurrentInstance().addMessage(null,  
        new FacesMessage("Completed tasks cleared"));  
}
```

Add a button to call this method.

Bonus 3 – Add Priority Counts to the Summary Page

Display:

- number of Low priority tasks
- number of Medium priority tasks
- number of High priority tasks

Hint: create methods in the bean similar to:

```
public long getHighPriorityTaskCount() {  
    return tasks.stream()  
        .filter(task -> "High".equals(task.getPriority()))  
        .count();  
}
```

Important Reminder

This practice is designed to help you prepare for Assignment 1. Do not just copy the code once and stop. Make small changes, break things, fix errors, and test again. That is how you build confidence with JSF.